

Practice Set for Mid-Sem

Section-1: MCQ / MSQ

(4 x 1 Marks = 4 Marks)

1. Consider the following two optimizers applied to the same model with the same initial weights and the same batch of data:

```
opt_A = torch.optim.SGD(model.parameters(), lr=0.01)
opt_B = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

After one parameter update step on the same gradient, which statement is true?

- (a) Both optimizers produce identical parameter updates on the first step because the momentum buffer is initialized to zero.
 - (b) opt_B produces the same update as opt_A on the very first step, but will produce different (typically larger) updates on subsequent steps as the momentum buffer accumulates.
 - (c) opt_B always produces larger updates than opt_A, even on the first step.
 - (d) Momentum only affects the learning rate schedule, not the gradient direction.
2. In a standard autoencoder with encoder f_θ and decoder g_ϕ , which loss function is most commonly used for reconstructing grayscale images with pixel values normalised to $[0, 1]$?
 - (a) Hinge loss
 - (b) Binary cross-entropy (BCE) loss
 - (c) Categorical cross-entropy loss
 - (d) Triplet loss
 3. (MSQ) Multiple Correct. Consider a CNN with the following architecture applied to a 1-channel 128×128 input image. More than one option may be correct. Select all that apply.

```
model = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, dilation=2, padding=2), # Layer 1
    nn.Conv2d(8, 16, kernel_size=3, dilation=1, padding=1), # Layer 2
    nn.Conv2d(16, 32, kernel_size=3, dilation=1, padding=1) # Layer 3
)
```

- (a) What is the effective receptive field of a single neuron in the output of Layer 1?

- i. 3×3
 - ii. 5×5
 - iii. 7×7
 - iv. 9×9
- (b) Which of the following statements are true about this network?
- i. The output spatial dimensions are 128×128 .
 - ii. Layer 1 increases the spatial resolution of the feature map.
 - iii. A neuron at the output of Layer 3 has a theoretical receptive field of 9×9 on the input image.
 - iv. Dilation in Layer 1 increases the number of learnable parameters compared to a standard 3×3 convolution.

Section-2: Short Answer Type Questions

(3 x 2 Marks = 6 Marks)

1. In the following code, what is the value of `y.grad` after execution?

```
x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(4.0, requires_grad=True)
z = x ** 2 + 2 * x * y + y ** 3
z.backward()
print(y.grad)
```

2. What are different techniques of image segmentation?
3. How does Backpropagation Through Time (BPTT) work in RNNs? What are the limitations of RNNs and how are they resolved?

Section-3: Long Answer Type Questions

(2 x 4 Marks = 8 Marks)

1. Predict the exact printed output of this code.

```
import torch
import torch.nn as nn

model = nn.Linear(1, 1, bias=True)
with torch.no_grad():
    model.weight.fill_(1.0)
    model.bias.fill_(0.5)

optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
```

```

x = torch.tensor([[2.0]])
target = torch.tensor([[3.0]])

optimizer.zero_grad()
pred = model(x)
loss = (pred - target) ** 2
loss.backward()

print("pred:", pred.item())
print("loss:", loss.item())
print("w.grad:", model.weight.grad.item())
print("b.grad:", model.bias.grad.item())

optimizer.step()
print("new_w:", model.weight.item())
print("new_b:", model.bias.item())

```

2. Consider the following CNN model defined in PyTorch. Carefully trace the tensor dimensions through each layer and predict the exact printed output.

```

import torch
import torch.nn as nn

class CustomCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=5, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.classifier = nn.Linear(32 * 6 * 6, 10)

    def forward(self, x):
        x = self.features(x)
        print("Feature shape:", x.shape)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        print("Output shape:", x.shape)
        return x

model = CustomCNN()
inp = torch.randn(4, 3, 32, 32)

with torch.no_grad():

```

```
out = model(inp)
```

- (a) What is the printed output of the code? Show step-by-step dimension calculations for each layer.
- (b) If the input image size were changed to 64×64 , what would the feature map dimensions be after the **features** block? Would the code crash? If yes, at which line and why?
- (c) What is the total number of learnable parameters (weights + biases) in this model?